

Experiment III — Continuous Integration for Infrastructure as Code

Field	Detail
Course outcome	CO5
Marks	5 marks (part of the 20 marks allotted to CO5)
Date of performance	_____
Marks awarded	_____

Aim

To implement continuous integration for Infrastructure as Code scripts by setting up a CI/CD pipeline and automating the testing and deployment of infrastructure changes.

Objectives

- To install and configure Jenkins and connect it to a Git repository containing Terraform code.
- To automate formatting, validation, linting and security scanning of infrastructure code.
- To automate deployment through a plan stage, a manual approval gate and an apply stage.
- To store cloud credentials securely so that no secret appears in the repository or the build log.
- To demonstrate that the pipeline correctly rejects defective and insecure infrastructure code.

Requirements

Hardware

- A machine with at least 4 GB RAM available for the Jenkins controller

Software

- Jenkins 2.4xx LTS
- Java 17 runtime (required by Jenkins)
- Terraform, tfint and tfsec installed on the Jenkins agent
- Git and Docker (optional, for running Jenkins as a container)

Accounts and access

- A GitHub or GitLab account hosting the Experiment II repository
- The AWS IAM credentials from Experiment I, to be stored in the Jenkins credential store

Theory

3.1 Why Infrastructure Code Needs a Pipeline

3.2 Describe Shift-Left Testing

3.3 Describe Static Analysis Tools for Terraform

3.4 Discuss the Plan Artefact and the Approval Gate

3.5 What is the Credential Handling in Jenkins

3.6 What do you mean by Branch-Based Promotion

Procedure

Step 1 — Install Jenkins

Install Jenkins from the official repository or run the LTS container image. Unlock the instance with the initial administrator password, install the suggested plugins and create an administrator account.

Running Jenkins as a container and retrieving the unlock password

```
$ docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
-v jenkins_home:/var/jenkins_home jenkins/jenkins:lts-jdk17

$ docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
8f3c1a94b26d4e7f9c05a1b2d3e4f5a6

# Open http://localhost:8080 and complete the setup wizard.
```

Step 2 — Install the required plugins

From Manage Jenkins → Plugins, install Pipeline, Git, Credentials Binding, AnsiColor and Blue Ocean. Restart Jenkins when installation completes.

Step 3 — Install the infrastructure tools on the agent

Terraform, tflint and tfsec must be present on the machine that executes the build, and must be reachable from the PATH of the user that Jenkins runs as.

Installing and verifying the toolchain on the agent

```
$ curl -fsSL https://raw.githubusercontent.com/terraform-linters/\
tflint/master/install_linux.sh | bash
$ curl -fsSL https://raw.githubusercontent.com/aquasecurity/tfsec/\
master/scripts/install_linux.sh | bash

$ terraform -version && tflint --version && tfsec --version
Terraform v1.9.5
TFLint version 0.52.0
tfsec v1.28.6
```

Step 4 — Store the cloud credentials

Navigate to Manage Jenkins → Credentials → System → Global credentials and add two Secret text credentials with the identifiers `aws-access-key-id` and `aws-secret-access-key`. Record only the identifiers in the record; never the values.

Step 5 — Create the pipeline job

Create a new item of type Multibranch Pipeline. Add the Git repository as a branch source, set the build configuration mode to "by Jenkinsfile" and the script path to Jenkinsfile. Jenkins will scan the repository and create one job per branch.

Step 6 — Author the Jenkinsfile

Commit the declarative pipeline shown in Listing 3.1 to the root of the repository. Each stage corresponds to one control from the theory section, in shift-left order.

Step 7 — Configure automatic triggering

Add a webhook in the Git host pointing at the Jenkins endpoint so that a push starts a build immediately. If the Jenkins instance is not reachable from the internet, configure SCM polling as a fallback.

Webhook endpoint and polling fallback

```
# GitHub → Settings → Webhooks → Add webhook
Payload URL:  http://<jenkins-host>:8080/github-webhook/
Content type: application/json
Events:       Just the push event

# Fallback when Jenkins is not publicly reachable, in the Jenkinsfile:
triggers { pollSCM('H/5 * * * *') }
```

Step 8 — Prove that the pipeline rejects badly formatted code

Deliberately break the indentation of a .tf file, commit and push. The build must fail at the Validate stage with a non-zero exit from `terraform fmt -check`, and the failing file must be named in the console output.

Step 9 — Prove that the pipeline rejects insecure code

Temporarily widen the SSH ingress rule to 0.0.0.0/0, commit and push. The build must fail at the Security Scan stage with a HIGH severity finding from tfsec. Record the rule identifier reported.

Step 10 — Run the successful path

Revert both defects, push, and observe the build progress through Validate, Security Scan and Plan. Download the archived plan artefact, review it, then approve the input step and let the Apply stage run.

Step 11 — Confirm secret masking and clean up

Open the full console log and search for the access key value; every occurrence must appear as asterisks. Finally, run a manual destroy job or execute `terraform destroy` locally so that no infrastructure is left running.

Program and Configuration Listings

Listing 3.1 — Jenkinsfile

```
pipeline {
  agent any

  options {
    ansiColor('xterm')
    timestamps()
    buildDiscarder(logRotator(numToKeepStr: '20'))
  }

  environment {
    AWS_ACCESS_KEY_ID      = credentials('aws-access-key-id')
    AWS_SECRET_ACCESS_KEY  = credentials('aws-secret-access-key')
    AWS_DEFAULT_REGION     = 'ap-south-1'
    TF_IN_AUTOMATION       = 'true'
  }

  stages {

    stage('Checkout') {
      steps { checkout scm }
    }

    stage('Validate') {
      steps {
        sh 'terraform fmt -check -recursive -diff'
        sh 'terraform init -input=false'
        sh 'terraform validate'
      }
    }

    stage('Security Scan') {
      steps {
        sh 'tflint --init && tflint --format compact'
        sh 'tfsec . --format junit --out tfsec-report.xml --soft-fail'
        sh 'tfsec . --minimum-severity HIGH' // fails the build on HIGH
      }
      post {
        always { junit allowEmptyResults: true, testResults: 'tfsec-report.xml' }
      }
    }

    stage('Plan') {
      steps {
        sh 'terraform plan -input=false -out=tfplan'
        sh 'terraform show -no-color tfplan > tfplan.txt'
        archiveArtifacts artifacts: 'tfplan, tfplan.txt', fingerprint: true
      }
    }

    stage('Approval') {
      when { branch 'main' }
      steps {
        timeout(time: 30, unit: 'MINUTES') {
          input message: 'Apply the archived plan to the cloud account?',
                 ok: 'Apply'
        }
      }
    }

    stage('Apply') {
      when { branch 'main' }
      steps { sh 'terraform apply -input=false tfplan' }
    }
  }

  post {
    success { echo 'Pipeline completed successfully.' }
    failure { echo 'Pipeline failed – inspect the stage that went red.' }
  }
}
```

```

    always { cleanWs() }
  }
}

```

Listing 3.2 — .tflint.hcl

```

plugin "aws" {
  enabled = true
  version = "0.32.0"
  source  = "github.com/terraform-linters/tflint-ruleset-aws"
}

rule "terraform_required_version" { enabled = true }
rule "terraform_required_providers" { enabled = true }
rule "terraform_naming_convention" { enabled = true }
rule "aws_instance_invalid_type" { enabled = true }

```

Expected Output

Output 3.1 — build failing at the Validate stage

```

[Pipeline] stage
[Pipeline] { (Validate)
+ terraform fmt -check -recursive -diff
modules/network/main.tf
--- old/modules/network/main.tf
+++ new/modules/network/main.tf
@@ -8,7 +8,7 @@
-   cidr_block=var.public_subnet_cidr
+   cidr_block = var.public_subnet_cidr

script returned exit code 3
[Pipeline] }
ERROR: script returned exit code 3
Finished: FAILURE

```

Output 3.2 — build failing at the Security Scan stage

```

[Pipeline] { (Security Scan)
+ tfsec . --minimum-severity HIGH

Result #1 HIGH  Security group rule allows ingress from the public internet
on port 22.

modules/network/main.tf:22-28
22   ingress {
23     from_port    = 22
26     cidr_blocks = ["0.0.0.0/0"]
28   }

ID          aws-ec2-no-public-ingress-sgr
Impact      Your port is exposed to the internet
Resolution  Set a more restrictive CIDR range

timings      disk i/o 118.4µs   parsing 51.2ms   evaluation 3.1ms
counts      files read 7       blocks 34       results 1
results     critical 0   high 1   medium 0   low 0

script returned exit code 1
Finished: FAILURE

```

Output 3.3 — successful pipeline with approval

```
Started by GitHub push by student-rollno
[Pipeline] { (Checkout)      -- 2 s  SUCCESS
[Pipeline] { (Validate)      -- 9 s  SUCCESS
[Pipeline] { (Security Scan) -- 6 s  SUCCESS (0 high, 0 critical)
[Pipeline] { (Plan)          -- 14 s SUCCESS
  Plan: 11 to add, 0 to change, 0 to destroy.
  Archiving artifacts: tfplan, tfplan.txt
[Pipeline] { (Approval)
  Apply the archived plan to the cloud account? [Apply] [Abort]
  Approved by admin
[Pipeline] { (Apply)          -- 48 s SUCCESS
  Apply complete! Resources: 11 added, 0 changed, 0 destroyed.

AWS_ACCESS_KEY_ID = ****                      (masked by Credentials Binding)
Finished: SUCCESS
```

Observation Table

Complete this table with the values observed during your own session.

Scenario committed	Stage that reacts	Expected behaviour	Observed
Badly indented .tf file	Validate		Exit code 3, build red
Missing required argument	Validate		Build red at validate
SSH open to 0.0.0.0/0	Security Scan		aws-ec2-no-public-ingress-sgr
Valid change, feature branch	Plan		Pipeline ended after Plan
Valid change, main branch	Approval		Waited for approval
Approval granted	Apply		11 resources added
New push to repository	Trigger		Started within 4 seconds
Secret in console log	All stages		Displayed as ****

Result

A multi-stage Jenkins pipeline was implemented for the Terraform project of Experiment II. The pipeline automatically validates, lints and security-scans every commit, produces a reviewable plan artefact, gates deployment behind human approval on the main branch only, and masks all credentials. It was demonstrated to reject both malformed and insecure infrastructure code, and to deploy correct code successfully.

Precautions

- Never place credentials in the Jenkinsfile or in any file committed to the repository. Use the credential store and reference credentials by identifier.
- Ensure the Apply stage is restricted to the main branch. A pipeline that applies from any branch removes the benefit of code review entirely.
- Pass the archived plan file to apply rather than letting apply recompute the plan, so that only the reviewed changes are executed.

- Configure a timeout on the input step, otherwise an unattended build will hold an executor indefinitely.
- If a credential ever appears unmasked in a build log, rotate it immediately; it must be treated as compromised.
- Run cleanWs() in the post block so that a working directory containing state or plan files is not left on the agent.

Viva Questions with Answers

Q1. What is the difference between continuous integration and continuous deployment?

Ans.

Q2. Why does the pipeline archive the plan file instead of recomputing it?

Ans.

Q3. How does Jenkins prevent credentials from leaking into build output?

Ans.

Q4. What is the difference between tflint and tfsec?

Ans.

Q5. Why are the stages ordered validate, scan, plan, apply?

Ans

Q6. How would you extend this pipeline for multiple environments?

Ans.

Q7. Why must a pipeline be tested with deliberately bad code?

Ans

Evaluation Rubric

#	Evaluation criterion	Marks	CO
1	Jenkins installed, configured and job created with secure credential storage	1	CO5
2	Jenkinsfile with correct multi-stage automated testing of the IaC code	1.5	CO5
3	Automated deployment with plan artefact, approval gate and successful apply	1.5	CO5
4	Failure scenarios demonstrated, record and viva	1	CO5
	TOTAL	5	CO5

Signature of the faculty in-charge: _____